A Minimal Calculus of Implicit Implementations

Scala 3 reference model 30 October 2026

1 Types, expressions, and contexts

$$\begin{array}{rcl} A, B, P, Q & ::= & \alpha \mid \mathsf{Int} \mid \mathsf{Bool} \mid \mathsf{String} \mid A \to B \mid A \Rightarrow B \mid \forall \alpha.A \mid \{\ell_j : A_j\}_{j \in J} \\ \Gamma & ::= & \cdot \mid \Gamma, \alpha \mid \Gamma, x : A \\ \Delta & ::= & [\,] \mid (d : A) :: \Delta \end{array}$$

Γ binds type variables and ordinary values; Δ binds available implementation values, nearest first. P, Q are ordinary types, not a distinct predicate sort. Records are finite maps with distinct labels; field order is immaterial. Equality is structural, modulo α-equivalence and lexical type-synonym expansion. There is no subtyping.

Expression notation for the typing rules (with resolved annotation types):

$$\begin{array}{rcl} e & ::= & x \mid d \mid n \mid b \mid s \mid \lambda(x : A).\, e \mid \lambda\{d : A\}.\, e \mid \lambda x.e \mid e(e) \mid e\{e\} \\ & & \mid \Lambda\alpha.e \mid e[A] \mid \mathsf{let}\ x = e\ \mathsf{in}\ e \mid \mathsf{impl}\ d : A = e\ \mathsf{in}\ e \mid \mathsf{fix}\ x : A.\, e \\ & & \mid \{\ell_j = e_j\}_{j \in J} \mid e.\ell \mid (e : A) \mid \mathsf{need}[A] \mid e_1 \odot e_2 \end{array}$$

The surface program has separate declarations and expressions, as specified in Section 2. Elaborating its declarations expands type and trait names and constructs a single unresolved term. The notation above describes the common binding and typing operations used in that translation. The syntax $e.\ell$ is an unclassified selection with an arbitrary term receiver. Elaboration determines whether a selection becomes a core projection or a method applied to its receiver. Both x and d are parsed as names; lexical name resolution distinguishes their binders. In rules, bound names are globally fresh by α-renaming. This convention preserves ordinary source shadowing without conflating the two environments.

Implicit terms:

$$\begin{array}{rcl} c & ::= & x \mid d \mid n \mid b \mid s \mid \lambda(x : A).\, c \mid \lambda\{d : A\}.\, c \mid c\, c \mid c\{c\} \\ & & \mid \Lambda\alpha.c \mid c[A] \mid \mathsf{let}\ x : A = c\ \mathsf{in}\ c \mid \mathsf{impl}\ d : A = c\ \mathsf{in}\ c \mid \mathsf{fix}\ x : A.\, c \\ & & \mid \{\ell_j = c_j\}_{j \in J} \mid c.\ell \mid ?_A \mid c_1 \odot c_2 \end{array}$$

Implicit terms contain no aliases, traits, definitions, modules, or method-resolution operation. $?_A$ is a deferred request, not a runtime term.

Primitive binary operators:

$\odot$	$\arg(\odot)$	$\mathrm{res}(\odot)$
$+, -, \times$	Int	Int
$==, <, >$	Int	Bool
$++$	String	String

$$\begin{array}{lll} \delta_+(m,n) = m+n, & \delta_-(m,n) = m-n, & \delta_\times(m,n) = mn, \\ \delta_{==}(m,n) = (m=n), & \delta_<(m,n) = (m<n), & \delta_>(m,n) = (m>n), \\ \delta_{++}(s,t) = st. & & \end{array}$$

Here $b \in \{\mathsf{true}, \mathsf{false}\}$; comparisons produce Boolean values.

Type formation (contexts are assumed well formed):

$$\frac{\alpha \in \Gamma}{\Gamma \vdash \alpha\ \mathsf{type}}\ \text{W-Var} \qquad \frac{B \in \{\mathsf{Int}, \mathsf{Bool}, \mathsf{String}\}}{\Gamma \vdash B\ \mathsf{type}}\ \text{W-Base}$$

$$\frac{\Gamma \vdash A\ \mathsf{type} \qquad \Gamma \vdash B\ \mathsf{type} \qquad \star \in \{\to, \Rightarrow\}}{\Gamma \vdash A \star B\ \mathsf{type}}\ \text{W-Arrow} \qquad \frac{\Gamma, \alpha \vdash A\ \mathsf{type} \qquad \alpha\ \mathsf{fresh}}{\Gamma \vdash \forall\alpha.A\ \mathsf{type}}\ \text{W-All}$$

$$\frac{\Gamma \vdash A_j\ \mathsf{type} \quad (j \in J) \qquad \ell_j\ \text{distinct}}{\Gamma \vdash \{\ell_j : A_j\}_{j \in J}\ \mathsf{type}}\ \text{W-Record}$$

2 Practical syntax and the first AST

Parsing produces a program consisting of top-level declarations followed by one final expression. The declaration forms are `type`, `trait`, `implement`, `instance`, and `def`:

$$\begin{aligned}
&\Pi = (\bar{D}, \hat{e}),\\
&D ::= \mathsf{type}\ C[\bar{a}] = \hat{A} \mid \mathsf{trait}\ C[\bar{a}] = \hat{A}\\
&\quad \mid \mathsf{implement}\ H\,\pi = \hat{e} \mid \mathsf{instance}\ d\,H : \hat{A} = \hat{e}\\
&\quad \mid \mathsf{def}\ f\,H(\bar{p}) : \hat{A} = \hat{e}.
\end{aligned}$$

Here H is an optional type parameter clause, π a trait reference, and $\bar{p}$ a parameter list. Instance parameter lists are expanded as described below. Write $\hat{e}$ for a parsed surface expression and $\hat{A}$ for a named surface type. Expressions include ordinary let bindings and local evidence bindings, each with a body:

$$\begin{aligned}
\texttt{let x = e in b} &\longmapsto \mathsf{let}\ x = \hat{e}\ \mathsf{in}\ \hat{b},\\
\texttt{let x: A = e in b} &\longmapsto \mathsf{let}\ x = (\hat{e} : \hat{A})\ \mathsf{in}\ \hat{b},\\
\texttt{given d: A = e in b} &\longmapsto \mathsf{impl}\ d : \hat{A} = \hat{e}\ \mathsf{in}\ \hat{b}.
\end{aligned}$$

A `given` requires its annotation; an ordinary let may synthesize the initializer's type. Neither binder is in scope in its own initializer. Braces and parentheses only group expressions. Declarations precede the final expression and cannot occur inside it. Elaboration consumes the declarations and produces one implicit term; type and trait declarations leave no runtime node.

Types and expression forms

Source type	Named abstract type
`Int`, `String`, `Bool`	$\mathsf{Int},\ \mathsf{String},\ \mathsf{Bool}$
C, $C[A_1, \ldots, A_n]$	C, $C[\hat{A}_1, \ldots, \hat{A}_n]$
`A -> B`	$\hat{A} \to \hat{B}$
`A ~> B`	$\hat{A} \Rightarrow \hat{B}$
`forall` $a_1, \ldots, a_n$. B	$\forall a_1. \cdots \forall a_n. \hat{B}$
$\{\ell_1 : A_1, \ldots, \ell_n : A_n\}$	$\{\ell_1 : \hat{A}_1, \ldots, \ell_n : \hat{A}_n\}$
(A)	$\hat{A}$

Both arrows associate to the right. A type name is retained as written; elaboration determines whether it denotes a bound type variable or a lexical synonym. A trait name is rejected in every ordinary type position, including annotations, type arguments, `summon`, and explicitly written implicit parameters.

Names and literals translate to x, n, b, and s; string escapes are decoded. The remaining nonbinding forms translate as follows:

Source expression	Named abstract expression
$\{\ell_1 = e_1, \ldots, \ell_n = e_n\}$	$\{\ell_1 = \hat{e}_1, \ldots, \ell_n = \hat{e}_n\}$
`e: A`	$(\hat{e} : \hat{A})$
`summon[A]`	$\mathsf{need}[\hat{A}]$
`e.field`	$\hat{e}.\mathit{field}$
`P::field`	$\hat{P} :: \mathit{field}$
`f(a)`, `f a`	$\hat{f}(\hat{a})$
`f(using a)`	$\hat{f}\{\hat{a}\}$
`f[A]`	$\hat{f}[\hat{A}]$
$e_1 \odot e_2$	$\hat{e}_1 \odot \hat{e}_2$
`(e)`, `{ e }`	$\hat{e}$

Field assignments distinguish a record literal from a brace group; `{}` is the empty record. Duplicate labels are rejected before fields become a finite map. A grouped expression can contain nested let and given expressions.

Consequently, `f { e }` translates to the ordinary application $\hat{f}(\hat{e})$. Binary operators use the primitive operations specified in Section 1.

Binders and application lists

A lambda parameter group contains one or more comma-separated entries `x`, `x: A`, or `implicit d: P`. Ordinary lambda annotations are optional; implicit parameters require annotations. Declaration parameters in `def` and `instance` require annotations. Concatenate repeated groups in source order to obtain $\bar{p} = [p_1, \ldots, p_n]$. Each entry wraps an abstract body E:

$$\begin{aligned}
\mathcal{L}_{x}(E) &= \lambda x.E, \\
\mathcal{L}_{x:A}(E) &= \lambda(x : \hat{A}).\, E, \\
\mathcal{L}_{\texttt{implicit } d:P}(E) &= \lambda\{d : \hat{P}\}.\, E, \\
\mathcal{L}_{\bar{p}}(E) &= \mathcal{L}_{p_1}(\mathcal{L}_{p_2}(\cdots \mathcal{L}_{p_n}(E)\cdots)).
\end{aligned}$$

A lambda `(`$\bar{p}$`) => e` translates to $\mathcal{L}_{\bar{p}}(\hat{e})$. A single unannotated parameter also admits `x => e`, producing $\lambda x.\hat{e}$. Comma-separated parameters and repeated groups therefore produce the same nested binders, with the first parameter outermost. For example:

$$\texttt{(a: A, implicit b: B, c: C) => e} \quad\longmapsto\quad \lambda(a : \hat{A}).\, \lambda\{b : \hat{B}\}.\, \lambda(c : \hat{C}).\, \hat{e}.$$

Type lambdas `[`$a_1, \ldots, a_n$`] => e` translate to $\Lambda a_1.\cdots\Lambda a_n.\hat{e}$.

Argument lists instead accumulate applications from left to right. Each `using` marks just its own argument; type-argument lists likewise produce successive type applications:

$$\begin{aligned}
\texttt{f(a, using b, c)} &\longmapsto ((\hat{f}(\hat{a}))\{\hat{b}\})(\hat{c}), \\
\texttt{f[A, B]} &\longmapsto (\hat{f}[\hat{A}])[\hat{B}].
\end{aligned}$$

Automatic insertion of missing implicit arguments belongs to elaboration of ordinary application; the parser preserves the applications actually written.

Declarations, programs, and type parameters

A file is a declaration sequence followed by a final expression:

$$\texttt{d}_1\texttt{;}\cdots\texttt{;d}_n\texttt{;e} \quad\longmapsto\quad ([\hat{d}_1, \ldots, \hat{d}_n], \hat{e}).$$

Here `;` stands for `in`, a semicolon, or a newline. The same separators join a let or given initializer to its body.

A `type C[`$\bar{a}$`] = A` declaration introduces an ordinary alias; `trait C[`$\bar{a}$`] = A` introduces a trait whose definition must expand to a record type. Both are sequential and nonrecursive. Parameters are distinct and the definition expands earlier aliases at declaration time. Its instantiated body is obtained by capture-avoiding substitution for those parameters. The frontend records the declaration kind, while the resulting calculus types are structural.

Write $\pi = C[A_1, \ldots, A_m]$ for a trait reference and Σ for the sequential declaration environment. The expansion $[\![\pi]\!]_\Sigma = D$ is defined only when C names a trait, the arity matches, and every A_i is an ordinary type. D is the dictionary's record type. Trait references occur in `where`, `implement` targets, and qualified member access. Explicit dictionary values use ordinary record types or aliases.

A type parameter clause has the form

$$H = \texttt{[}a_1, \ldots, a_n\ \texttt{where}\ \pi_1\ \texttt{and}\ \cdots\ \texttt{and}\ \pi_k\texttt{]}.$$

The type parameter list may be empty. The optional `where` clause, when present, contains one or more trait references separated by `and`. Type parameter clauses belong to `def`, `implement`, and `instance` declarations. After binding the type parameters, elaboration resolves $D_j = [\![\pi_j]\!]_\Sigma$ and uses these abstraction operations:

$$\begin{aligned}
\mathcal{G}_H(c) &= \Lambda a_1.\cdots\Lambda a_n.\lambda\{d_1 : D_1\}.\ \cdots\lambda\{d_k : D_k\}.\, c, \\
\mathcal{Q}_H(T) &= \forall a_1.\cdots\forall a_n.D_1 \Rightarrow \cdots \Rightarrow D_k \Rightarrow T.
\end{aligned}$$

The d_j are fresh source-inaccessible binders. An omitted clause makes both operations the identity.

The two evidence declarations are

```
implement[A where C[A]] D[A] = expression
instance name[A where C[A]](parameters): R = expression
```

Type parameter clauses are optional; instance value parameters are also optional. The first form requires a trait target and is anonymous. The second requires a name and accepts any ordinary type. Both are top-level and nonrecursive.

Instance value parameters use the same curried binders as lambdas. Their annotations also construct a function type, with each implicit parameter choosing $\Rightarrow$:

$$\mathcal{F}_{x:A}(T) = \hat{A} \to T, \qquad \mathcal{F}_{\texttt{implicit } d:P}(T) = \hat{P} \Rightarrow T,$$
$$\mathcal{F}_{\bar{p}}(T) = \mathcal{F}_{p_1}(\cdots \mathcal{F}_{p_n}(T) \cdots).$$

Thus `instance` $\texttt{d}H\texttt{(}\bar{p}\texttt{):}$ `R = e` becomes the declaration $\mathsf{instance}\ d\,H : \mathcal{F}_{\bar{p}}(\hat{R}) = \mathcal{L}_{\bar{p}}(\hat{e})$. With no value parameters, these operations are the identity.

If T is an implementation's expanded trait type or an instance's resolved annotation, the initializer is checked against T under the clause's type and constraint binders. Let c be that checked initializer and b the elaboration of the remaining program. Both evidence declarations produce

$$\mathsf{impl}\ d : \mathcal{Q}_H(T) = \mathcal{G}_H(c)\ \mathsf{in}\ b.$$

The new evidence binding is available only in b. Its name is supplied by `instance` or generated for `implement`. A local `given d: A = e in b` instead checks e against its ordinary annotation directly, then elaborates b with $d : A$ in Δ. A polymorphic local value uses an explicit universal annotation and type abstraction, for example `given id: forall A. A -> A = [A] => x => x in body`. Type and trait declarations update Σ and contribute no term binding.

Definitions and recursion

A definition retains its name, type parameters, value parameters, result annotation, and body:

```
def f[A where C[A]](parameters): R = body
```

The type parameter clause and value parameter groups are optional; the result annotation is required. Let V be the body abstracted over the written parameters and type parameter clause, and T its full declared type, including those arrows and quantifiers. The generated `where` parameters precede the written parameters. A definition with no self-reference produces $\mathsf{let}\ f : T = V\ \mathsf{in}\ b$. Otherwise the body is checked with $f : T$ in Γ and produces

$$\mathsf{let}\ f : T = \mathsf{fix}\ f : T.\, V\ \mathsf{in}\ b.$$

A self-reference is a free term occurrence of f in the abstracted body: parameter, lambda, let, and given shadowing are respected. A local binding does not bind its own initializer. Field labels and type names are not term references. Declarations are sequential; the self binding does not enable mutual or forward recursion.

The surface restriction on recursive definitions is

$$\mathrm{function}(A \to B) = \mathsf{true}, \qquad \mathrm{function}(A \Rightarrow B) = \mathsf{true},$$
$$\mathrm{function}(\forall \alpha.A) = \mathrm{function}(A), \qquad \mathrm{function}(A) = \mathsf{false} \quad \text{otherwise}.$$

Elaboration requires $\mathrm{function}(T)$ only when inserting a fixpoint. Thus `def x: Int = x` and a self-referencing record definition are rejected, while nonrecursive values of those types are allowed. The calculus's fixpoint has no function-type restriction. Neither the core nor the surface function check promises termination.

For example, a nonrecursive definition

```
def display[A where ToString[A]](x: A): String = x.toString
rest
```

remains a definition declaration followed by the expression `rest` in the surface program. If $D_A = \{\mathsf{toString} : A \to \mathsf{String}\}$, elaboration produces

$$\mathsf{let\ display} : \forall A.D_A \Rightarrow A \to \mathsf{String} = \Lambda A.\lambda\{d : D_A\}.\, \lambda(x : A).\, (?_{D_A}).\mathsf{toString}\ x\ \mathsf{in}\ c_{\mathsf{rest}}.$$

This is one term: the trait and definition declarations have been consumed by elaboration.

Grouping and the elaboration boundary

Precedence, from tightest to loosest, is adjacent postfix operations, application, `*`, `+`/`-`, `++`, comparisons, and ascription. Applications and binary operators associate to the left. Adjacency determines the argument tree: `f g(x)` parses as `f(g(x))`, while `f g (x)` parses as `f(g)(x)`. A negative argument is parenthesized, as in `f(-1)`; `f - 1` is subtraction.

A newline after a complete expression ends its application, selection, ascription, and operator chain, allowing it to separate a declaration initializer from its continuation. Line breaks are allowed where a component is still required, such as after `=` or `=>`. Record fields require commas, semicolons, or newlines, with an optional trailing separator. Comments are whitespace; newlines inside them also act as boundaries.

The program records declaration order; expression binders record lexical nesting. Elaboration resolves term names against ordinary and implementation binders, resolves type names against type variables and synonyms, validates trait references, and checks annotations. It classifies each selection $e.\ell$ using the receiver's type: an existing record field becomes a projection; otherwise the method rule may introduce an implementation request and apply its selected member to the receiver. For `P::field`, elaboration first resolves P as a trait reference or ordinary type, obtaining a dictionary type D, then elaborates $(\texttt{need}[D]).\mathit{field}$. Types and traits remain separate from term receivers: `ToString[A].toString` fails elaboration when `ToString` names only a trait. Evidence resolution later supplies implementations for the resulting requests.

Worked translations

The following complete programs illustrate how declaration elaboration and expression translation compose. Symbols such as v and e_{body} abbreviate subexpressions of the displayed mathematical tree. Generated binders are fresh and source-inaccessible.

1. Type abstractions with mixed parameters and arguments.

```
trait Default[A] = { value: A }
def choose[A where Default[A]](fallback: A, implicit preferred: A)(last: A): A = preferred
implement Default[Int] = { value = 11 }
choose[Int](0, using 21, 99)
```

The definition introduces its type parameter, its generated constraint parameter, and then its three written parameters, in that order. After validating and expanding the trait:

$$v_{\mathsf{choose}} = \Lambda A.\lambda\{d : \{\mathsf{value} : A\}\}.\,\lambda(\mathsf{fallback} : A).\,\lambda\{\mathsf{preferred} : A\}.\,\lambda(\mathsf{last} : A).\,(\mathsf{preferred} : A),$$
$$e = \mathsf{let\ choose} = v_{\mathsf{choose}}\ \mathsf{in\ impl}\ i : \{\mathsf{value} : \mathsf{Int}\} = \{\mathsf{value} = 11\}\ \mathsf{in}\ ((\mathsf{choose}[\mathsf{Int}](0))\{21\})(99).$$

Only the argument 21 is marked implicit in the parsed call. When elaboration reaches the first ordinary argument 0, it inserts the request for the leading `where` constraint. The unresolved call is therefore

$$((\mathsf{choose}[\mathsf{Int}]\{?_{\{\mathsf{value:Int}\}}\})(0))\{21\}(99).$$

Resolution supplies $\{\mathsf{value} = 11\}$ for that request; 21 is supplied directly to preferred. The program returns 21. Replacing the two parameter groups by `(fallback: A)(implicit preferred: A)(last: A)` gives the same abstract definition.

2. Typed bindings, brace groups, and local implementations.

```
type Box[A] = { value: A }
let box: Box[Int] = {
  let seed: Int = 20
  given chosen: Int = seed + 1
  { value = summon[Int] }
}
box.value
```

The initializer's outer braces group nested let and given expressions; the inner field assignment constructs a record. After annotations and the alias are resolved:

$$e_{\text{body}} = \mathsf{let\ seed} : \mathsf{Int} = 20\ \mathsf{in\ impl\ chosen} : \mathsf{Int} = \mathsf{seed} + 1\ \mathsf{in}\ \{\mathsf{value} =?_{\mathsf{Int}}\},$$
$$e = \mathsf{let\ box} : \{\mathsf{value} : \mathsf{Int}\} = e_{\text{body}}\ \mathsf{in\ box.value}.$$

The newline after the closing brace starts the continuation of `let box`. The implementation is available while constructing the record; its scope ends before box.value. The record retains the value 21, which is the program's result.

3 Typing and elaboration: functions and bindings

Bidirectional judgments and name resolution

$$\boxed{\Gamma; \Delta \vdash e \Rightarrow A \rightsquigarrow c} \qquad \boxed{\Gamma; \Delta \vdash e \Leftarrow A \rightsquigarrow c}$$

The first judgment synthesizes A from e; the second checks e against an expected A. Both produce a fully annotated implicit term c. An unannotated lambda requires checking against an ordinary function type. Type annotations, application parameter types, definition signatures, and evidence declarations supply expected types. Checking propagates these types through nested lambdas, type abstractions, record fields, and binding bodies. Let initializers without annotations synthesize their types independently of later uses. No type is inferred backward from a lambda's body or from later calls.

Resolution, target typing, and evaluation use the separate judgments

$$\boxed{\Gamma; \Delta \vdash c \Longrightarrow t} \qquad \boxed{H \vdash t : T} \qquad \boxed{\rho \vdash t \Downarrow v}.$$

The implementation uses separate de Bruijn indices for Γ and Δ, and a type-variable depth. The final target has one term-index space.

A nonrecursive synonym $C[\bar{\alpha}] = B$ expands $C[\bar{A}]$ to $B[\bar{A}/\bar{\alpha}]$ in its lexical body. Expansion is capture avoiding: the declaration's parameters bind only their own occurrences in B. Earlier aliases are expanded when the declaration is checked. $|\bar{A}| = |\bar{\alpha}|$ is required. Types and values resolve in separate namespaces. All annotation and expected types must be well formed. These judgments do not search for evidence; they insert deferred requests.

$$\frac{x : A \in \Gamma}{\Gamma; \Delta \vdash x \Rightarrow A \rightsquigarrow x}\ \textsc{E-Var} \qquad \frac{d : A \in \Delta}{\Gamma; \Delta \vdash d \Rightarrow A \rightsquigarrow d}\ \textsc{E-Instance}$$

$$\frac{\Gamma, x : A; \Delta \vdash e \Rightarrow B \rightsquigarrow c}{\Gamma; \Delta \vdash \lambda(x : A).\, e \Rightarrow A \to B \rightsquigarrow \lambda(x : A).\, c}\ \textsc{E-Lambda}$$

$$\frac{\Gamma, x : A; \Delta \vdash e \Leftarrow B \rightsquigarrow c}{\Gamma; \Delta \vdash \lambda x.e \Leftarrow A \to B \rightsquigarrow \lambda(x : A).\, c}\ \textsc{Check-Lambda}$$

$$\frac{\Gamma, x : A; \Delta \vdash e \Leftarrow B \rightsquigarrow c}{\Gamma; \Delta \vdash \lambda(x : A).\, e \Leftarrow A \to B \rightsquigarrow \lambda(x : A).\, c}\ \textsc{Check-Annotated-Lambda}$$

$$\frac{\Gamma; (d : A) :: \Delta \vdash e \Rightarrow B \rightsquigarrow c}{\Gamma; \Delta \vdash \lambda\{d : A\}.\, e \Rightarrow A \Rightarrow B \rightsquigarrow \lambda\{d : A\}.\, c}\ \textsc{E-Implicit-Lambda}$$

$$\frac{\Gamma; (d : A) :: \Delta \vdash e \Leftarrow B \rightsquigarrow c}{\Gamma; \Delta \vdash \lambda\{d : A\}.\, e \Leftarrow A \Rightarrow B \rightsquigarrow \lambda\{d : A\}.\, c}\ \textsc{Check-Implicit-Lambda}$$

$$\frac{\Gamma, \alpha; \Delta \vdash e \Rightarrow A \rightsquigarrow c \qquad \alpha \notin \mathrm{FV}(\Gamma, \Delta)}{\Gamma; \Delta \vdash \Lambda\alpha.e \Rightarrow \forall\alpha.A \rightsquigarrow \Lambda\alpha.c}\ \textsc{E-Type-Lambda}$$

$$\frac{\Gamma, \alpha; \Delta \vdash e \Leftarrow A \rightsquigarrow c \qquad \alpha \notin \mathrm{FV}(\Gamma, \Delta)}{\Gamma; \Delta \vdash \Lambda\alpha.e \Leftarrow \forall\alpha.A \rightsquigarrow \Lambda\alpha.c}\ \textsc{Check-Type-Lambda}$$

$$\frac{\Gamma; \Delta \vdash e \Rightarrow \forall \alpha. A \rightsquigarrow c \qquad \Gamma \vdash B \text{ type}}{\Gamma; \Delta \vdash e[B] \Rightarrow A[B/\alpha] \rightsquigarrow c[B]} \text{ E-Type-App}$$

$$\frac{\Gamma; \Delta \vdash e_1 \Rightarrow A \Rightarrow B \rightsquigarrow c_1 \qquad \Gamma; \Delta \vdash e_2 \Leftarrow A \rightsquigarrow c_2}{\Gamma; \Delta \vdash e_1\{e_2\} \Rightarrow B \rightsquigarrow c_1\{c_2\}} \text{ E-Implicit-App}$$

$$\frac{\Gamma; \Delta \vdash e_1 \Rightarrow T \rightsquigarrow c_1 \qquad c_1 : T \triangleright c_1' : A \to B \qquad \Gamma; \Delta \vdash e_2 \Leftarrow A \rightsquigarrow c_2}{\Gamma; \Delta \vdash e_1(e_2) \Rightarrow B \rightsquigarrow c_1'\, c_2} \text{ E-App}$$

$$\frac{\Gamma; \Delta \vdash e_1 \Rightarrow A \rightsquigarrow c_1 \qquad \Gamma, x : A; \Delta \vdash e_2 \Rightarrow B \rightsquigarrow c_2}{\Gamma; \Delta \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \Rightarrow B \rightsquigarrow \mathsf{let}\ x : A = c_1\ \mathsf{in}\ c_2} \text{ E-Let}$$

$$\frac{\Gamma; \Delta \vdash e_1 \Rightarrow A \rightsquigarrow c_1 \qquad \Gamma, x : A; \Delta \vdash e_2 \Leftarrow B \rightsquigarrow c_2}{\Gamma; \Delta \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \Leftarrow B \rightsquigarrow \mathsf{let}\ x : A = c_1\ \mathsf{in}\ c_2} \text{ Check-Let}$$

$$\frac{\Gamma; \Delta \vdash e_1 \Leftarrow A \rightsquigarrow c_1 \qquad \Gamma; (d : A) :: \Delta \vdash e_2 \Rightarrow B \rightsquigarrow c_2}{\Gamma; \Delta \vdash \mathsf{impl}\ d : A = e_1\ \mathsf{in}\ e_2 \Rightarrow B \rightsquigarrow \mathsf{impl}\ d : A = c_1\ \mathsf{in}\ c_2} \text{ E-Impl}$$

$$\frac{\Gamma; \Delta \vdash e_1 \Leftarrow A \rightsquigarrow c_1 \qquad \Gamma; (d : A) :: \Delta \vdash e_2 \Leftarrow B \rightsquigarrow c_2}{\Gamma; \Delta \vdash \mathsf{impl}\ d : A = e_1\ \mathsf{in}\ e_2 \Leftarrow B \rightsquigarrow \mathsf{impl}\ d : A = c_1\ \mathsf{in}\ c_2} \text{ Check-Impl}$$

$$\frac{\Gamma \vdash A \text{ type} \qquad \Gamma, x : A; \Delta \vdash e \Leftarrow A \rightsquigarrow c}{\Gamma; \Delta \vdash \mathsf{fix}\ x : A.\, e \Rightarrow A \rightsquigarrow \mathsf{fix}\ x : A.\, c} \text{ E-Fix} \qquad \frac{\Gamma; \Delta \vdash e \Leftarrow A \rightsquigarrow c}{\Gamma; \Delta \vdash (e : A) \Rightarrow A \rightsquigarrow c} \text{ E-Ascribe}$$

$$\frac{\Gamma \vdash A \text{ type}}{\Gamma; \Delta \vdash \mathsf{need}[A] \Rightarrow A \rightsquigarrow ?_A} \text{ E-Need} \qquad \frac{\Gamma; \Delta \vdash e \Rightarrow A \rightsquigarrow c \qquad e \text{ is not a checking introduction}}{\Gamma; \Delta \vdash e \Leftarrow A \rightsquigarrow c} \text{ Check-Infer}$$

The checking introductions are ordinary, implicit, and type lambdas, records, lets, and implementation bindings. They use their corresponding checking rules, including Check-Record below. Every supplied lambda annotation must equal the expected parameter type. Checking does not insert an ordinary, implicit, or type abstraction; the source determines the binder kind.

Expose an ordinary application

$$\frac{T \neq (P \Rightarrow Q) \text{ for every } P, Q}{c : T \triangleright c : T} \text{ Expose-Stop} \qquad \frac{c\{?_P\} : Q \triangleright c' : T}{c : P \Rightarrow Q \triangleright c' : T} \text{ Expose-Step}$$

Only ordinary application invokes this relation. In particular, returning an implicit function, referring to it by name, or passing it as an ordinary value does not force it. An implementation initializer is elaborated in the old Δ: bindings are nonrecursive.

4 Typing and elaboration: records, primitives, and method syntax

$$\frac{n \in \mathbb{Z}}{\Gamma; \Delta \vdash n \Rightarrow \mathsf{Int} \rightsquigarrow n}\ \text{E-Int} \qquad \frac{s \text{ a string}}{\Gamma; \Delta \vdash s \Rightarrow \mathsf{String} \rightsquigarrow s}\ \text{E-String} \qquad \frac{b \in \{\mathsf{true}, \mathsf{false}\}}{\Gamma; \Delta \vdash b \Rightarrow \mathsf{Bool} \rightsquigarrow b}\ \text{E-Bool}$$

$$\frac{\Gamma; \Delta \vdash e_j \Rightarrow A_j \rightsquigarrow c_j \quad (j \in J) \qquad \ell_j \text{ distinct}}{\Gamma; \Delta \vdash \{\ell_j = e_j\}_{j \in J} \Rightarrow \{\ell_j : A_j\}_{j \in J} \rightsquigarrow \{\ell_j = c_j\}_{j \in J}}\ \text{E-Record}$$

$$\frac{\Gamma; \Delta \vdash e_j \Leftarrow A_j \rightsquigarrow c_j \quad (j \in J) \qquad \ell_j \text{ distinct}}{\Gamma; \Delta \vdash \{\ell_j = e_j\}_{j \in J} \Leftarrow \{\ell_j : A_j\}_{j \in J} \rightsquigarrow \{\ell_j = c_j\}_{j \in J}}\ \text{Check-Record}$$

$$\frac{\Gamma; \Delta \vdash e \Rightarrow \{\ell_j : A_j\}_{j \in J} \rightsquigarrow c \qquad k \in J}{\Gamma; \Delta \vdash e.\ell_k \Rightarrow A_k \rightsquigarrow c.\ell_k}\ \text{E-Project}$$

$$\frac{\Gamma; \Delta \vdash e_1 \Leftarrow \arg(\odot) \rightsquigarrow c_1 \qquad \Gamma; \Delta \vdash e_2 \Leftarrow \arg(\odot) \rightsquigarrow c_2}{\Gamma; \Delta \vdash e_1 \odot e_2 \Rightarrow \mathrm{res}(\odot) \rightsquigarrow c_1 \odot c_2}\ \text{E-BinOp}$$

Receiver-style method selection

For a receiver type A and label ℓ, define the finite set of *types*

$$\mathcal{M}_{\Delta,A,\ell} = \big\{(P, B) \;\big|\; (d : P) \in \Delta,\ P = \{\ell_j : A_j\}_{j \in J},\ \exists k \in J.\ \ell_k = \ell \wedge A_k = A \to B\big\}.$$

Repeated occurrences of the same pair contribute one set member. Let $\mathrm{labels}(A)$ be the field labels when A is a record, and $\varnothing$ otherwise.

$$\frac{\Gamma; \Delta \vdash e \Rightarrow A \rightsquigarrow c \qquad \ell \notin \mathrm{labels}(A) \qquad \mathcal{M}_{\Delta,A,\ell} = \{(P, B)\}}{\Gamma; \Delta \vdash e.\ell \Rightarrow B \rightsquigarrow (?_P).\ell\ c}\ \text{E-Method}$$

Zero candidates reject the selection; two distinct candidate types reject it as ambiguous. This rule determines the request type, not its implementation. Actual record projection has priority; records without the requested field can also use this rule. Explicit qualification $\mathsf{need}[P].\ell(e)$, written `P::member(e)` in source syntax, avoids this receiver-syntax restriction.

The running qualified function

With $\mathsf{ToString}[A] = \{\mathsf{toString} : A \to \mathsf{String}\}$, the source definition

```
def display[A where ToString[A]](x: A): String = x.toString
```

has type and unresolved body

$$\begin{aligned} &\mathsf{display} : \forall A.\mathsf{ToString}[A] \Rightarrow A \to \mathsf{String}, \\ &c_{\mathsf{display}} = \Lambda A.\lambda\{d : \mathsf{ToString}[A]\}.\,\lambda(x : A).\,(?_{\mathsf{ToString}[A]}).\mathsf{toString}\ x. \end{aligned}$$

Under $(d : \mathsf{ToString}[A]) :: \Delta$, evidence resolution may select d. After successful resolution:

$$t_{\mathsf{display}} = \Lambda A.\lambda(d : \mathsf{ToString}[A]).\,\lambda(x : A).\,d.\mathsf{toString}\ x, \qquad t_{\mathsf{display}} : \forall A.\mathsf{ToString}[A] \to A \to \mathsf{String}.$$

A concrete call first becomes $c_{\mathsf{display}}[\mathsf{String}]\{?_{\mathsf{ToString}[\mathsf{String}]}\}\ s$. The request is solved in the *caller's* lexical environment, while requests in an ordinary function body are solved in that body's definition environment.

5 Admissible evidence and the synthesizer

Evidence constructions are a restricted subset of implicit terms:

$$\boxed{p ::= d \mid p[A] \mid p\,p \mid p\{p\}.}$$

The only leaves are references into Δ. New literals, lambdas, projections, ordinary references from Γ, and unresolved requests are not synthesis results. They may occur inside a previously checked first-class value installed in Δ.

$$\frac{d : A \in \Delta}{\Gamma; \Delta \vdash_{\text{ev}} d : A}\ \text{P-Instance} \qquad \frac{\Gamma; \Delta \vdash_{\text{ev}} p : \forall\alpha.A \qquad \Gamma \vdash B\ \mathsf{type}}{\Gamma; \Delta \vdash_{\text{ev}} p[B] : A[B/\alpha]}\ \text{P-Type-App}$$

$$\frac{\Gamma; \Delta \vdash_{\text{ev}} p_1 : A \to B \qquad \Gamma; \Delta \vdash_{\text{ev}} p_2 : A}{\Gamma; \Delta \vdash_{\text{ev}} p_1\,p_2 : B}\ \text{P-App}$$

$$\frac{\Gamma; \Delta \vdash_{\text{ev}} p_1 : A \Rightarrow B \qquad \Gamma; \Delta \vdash_{\text{ev}} p_2 : A}{\Gamma; \Delta \vdash_{\text{ev}} p_1\{p_2\} : B}\ \text{P-Implicit-App}$$

These rules *check* a proposed construction. They do not prescribe a search strategy. A pluggable algorithm supplies a partial judgment

$$\Gamma; \Delta \vdash_{\mathcal{S}} A \rightsquigarrow p.$$

The resolver rechecks $\Gamma; \Delta \vdash_{\text{ev}} p : A$ before lowering. The source arrow kind is checked before erasure, even if two candidate types would erase to the same ordinary function type. Search uses Γ only for its rigid type variables; its ordinary term bindings do not supply evidence.

Exact-lookup baseline

Let $\Delta = [d_0 : A_0, \ldots, d_{m-1} : A_{m-1}]$, nearest binding first.

$$\frac{0 \le j < m \qquad A_j = A \qquad \forall k < j.\ A_k \neq A}{\Gamma; \Delta \vdash_{\text{naive}} A \rightsquigarrow d_j}\ \text{S-Exact}$$

If no j satisfies the premises, the baseline fails. It does not instantiate a universal type, apply a producer, recurse, or search for a hidden type argument. Failure is not an uninhabitability result.

Bounded SMT synthesis

The default synthesizer first uses S-Exact. Otherwise it searches application *spines*: a head reference followed by a sequence of eliminations. For bounds $N, H > 0$, a candidate program is

$$\begin{aligned} z_i &= d_{j_i}\, a_{i,1} \cdots a_{i,\ell_i} && (0 \le i \le r < N),\\ a_{i,t} &::= [B] \mid z_k \mid \{z_k\} && (0 \le k < i),\\ \sum_{i=0}^{r} (1 + \ell_i) &\le N. \end{aligned}$$

The z_i are notation for previously constructed evidence, not new source binders or additional members of Δ. Decoding expands these references. A shared argument is counted once in the program, even if decoding repeats its evidence tree. Every head must be an existing $d_{j_i} : A_{j_i} \in \Delta$.

Introduce symbolic types $T_{i,0}, \ldots, T_{i,\ell_i}$ for each spine. The initial constraint is $T_{i,0} = A_{j_i}$. Each elimination contributes one of the following constraints, writing $U_k = T_{k,\ell_k}$ for a completed earlier spine:

$a_{i,t}$	premises	$T_{i,t}$
$[B]$	$T_{i,t-1} = \forall\alpha.C, \quad \Gamma \vdash B\ \mathsf{type}, \quad h(B) \le H$	$C[B/\alpha]$
z_k	$T_{i,t-1} = U_k \to C$	C
$\{z_k\}$	$T_{i,t-1} = U_k \Rightarrow C$	C

Here h is constructor height: base types and variables have height one, arrows and universals add one to their children's maximum height, and a record has height one plus its deepest field type (the empty record has height one). The final constraint is $U_r = A$, the requested type. Both kinds of arrow remain distinct throughout this search.

The solver considers increasing numbers of spines, up to N, with the same total cost bound. A successful result need not minimize cost or select a unique proof among solutions with the same number of spines. Only the initial nearest-exact preference is a lexical priority rule; no general coherence property is claimed. The implementation defaults to $N = 16$, $H = 8$, and a ten-second search timeout. These limits are configurable and do not restrict explicit evidence written by the programmer.

Type encoding. Types are algebraic datatypes in SMT. At this encoding boundary, each unordered field map is serialized in increasing label order, so datatype equality coincides with map equality; decoding reconstructs an unordered map. Existing type variables are rigid de Bruijn indices, not unification metavariables; inferred type arguments are symbolic values of the type datatype. At ambient depth q, well-formedness requires $0 \le i < q$ for a variable $\underline{i}$ and increases q beneath $\forall$. The serialized labels must be distinct and increasing; ordering is an encoding invariant, not a property of calculus records. Type arguments may contain nested universals as well as either arrow.

Instantiation removes one binder and avoids capture. If $I_k(C, B)$ substitutes for the binder beneath k retained universals, its variable and universal cases are

$$I_k(\underline{i}, B) = \begin{cases} \underline{i} & i < k, \\ \uparrow_0^k B & i = k, \\ \underline{i-1} & i > k, \end{cases} \qquad I_k(\forall.C, B) = \forall.I_{k+1}(C, B).$$

The operation is homomorphic on the remaining constructors. The shift $\uparrow_0^k$ increases free indices while preserving indices bound within B. Thus instantiation is $I_0(C, B)$; an outer rigid variable never becomes a freshly inferred variable.

Record labels are encoded by their positions in the ordered vocabulary of labels occurring in A or Δ. This restriction preserves existence of bounded evidence: delete every other label from all types in a witness. This transformation leaves the input types unchanged, commutes with substitution and every evidence typing rule, and never increases type height or program cost. It introduces no new record values or projections into evidence.

Constraint construction and validation. A known implementation type determines its initial sequence of eliminations. Interpreting that known structure with symbolic type arguments avoids asking SMT to rediscover it. When an inferred type becomes the function type, its constructor and the next elimination remain symbolic, including possible value application followed by type application. This is partial evaluation of the same constraints, not a separate matching algorithm. When no earlier spine exists, value application is impossible and its continuation is omitted.

Well-formedness and height constraints are enforced by model refinement. An initial model is provisional. Every active type-argument position is checked; for each violation, the solver receives the general constraint that this position must be well formed and within height H whenever it is active. Search then resumes with the same spine count. Every added constraint is required by P-Type-App and excludes no valid witness. Each position can require this refinement at most once. Evidence is decoded only after all these obligations hold. Native solver state belongs to one request and is discarded after success or failure.

Guarantees and failures. Induction over the spine order, followed by its eliminations, derives $\Gamma; \Delta \vdash_{\mathrm{ev}} p : A$ for a satisfying, validated model. Conversely, flattening each function position in an admissible evidence tree and ordering its argument subtrees before their uses yields such a program. Therefore every witness whose program cost and type-argument heights meet the bounds is represented by the constraints. Solver timeouts or an *unknown* answer may still prevent discovery. The resolver independently rechecks the decoded construction and its erased target; an SMT model is not a trusted typing derivation.

An unsatisfiable bounded query normally reports *search limit exhausted*, not uninhabitability. There are two definite absence cases. Empty Δ admits no evidence leaves. Also, if neither the goal nor any type in Δ contains $\forall$, let k be the number of distinct subtypes of those inputs. Elimination can only produce these types. Choose a least-height proof for each inhabited type; its dependencies have strictly smaller least heights. There are at most k such proofs, and each head spine has at most k nodes. A program of cost at most k^2 therefore suffices whenever any proof exists. Unsatisfiability after covering that bound establishes absence. Otherwise exhaustion, solver unknown (including timeout), and native/API failures remain distinct outcomes, with the underlying reason preserved.

First-class polymorphic implementations

A type abstraction can provide an implementation of type

$$i : \forall A, B.\mathsf{Repr}[A, B] \Rightarrow \mathsf{ToString}[B] \Rightarrow \mathsf{ToString}[A], \qquad \mathsf{Repr}[A, B] = \{\mathsf{repr} : A \to B\}.$$

Given $r : \mathsf{Repr}[\mathsf{Person}, \mathsf{String}]$ and $s : \mathsf{ToString}[\mathsf{String}]$ in Δ, the construction

$$p = i[\mathsf{Person}][\mathsf{String}]\{r\}\{s\}$$

satisfies $\Gamma; \Delta \vdash_{\mathrm{ev}} p : \mathsf{ToString}[\mathsf{Person}]$. The SMT synthesizer discovers the intermediate type $B = \mathsf{String}$ and this composition; the exact-lookup baseline does not. The source program can also construct and install it explicitly:

```
instance person: { toString: Person -> String } =
  via[Person, String](using personName, using string)
ToString[Person]::toString({ name = "Ada" })
```

There is no special type of implementation values: i, r, s are ordinary values, and impl only changes which context binds them.

6 Resolution and lowering

Type erasure (identity on variables and base types):

$$|A \to B| = |A| \to |B|, \qquad |A \Rightarrow B| = |A| \to |B|,$$
$$|\forall \alpha.A| = \forall \alpha.|A|, \qquad |\{\ell_j : A_j\}_{j \in J}| = \{\ell_j : |A_j|\}_{j \in J}.$$

For evidence, $|d| = d$, $|p[A]| = |p|[|A|]$, and both evidence application forms erase to ordinary application. Ordinary and instance names are embedded in one target namespace; freshness prevents capture.

$$\frac{x : A \in \Gamma}{\Gamma; \Delta \vdash x \Longrightarrow x}\ \text{R-Var} \qquad \frac{d : A \in \Delta}{\Gamma; \Delta \vdash d \Longrightarrow d}\ \text{R-Instance} \qquad \frac{\Gamma; \Delta \vdash_{\mathcal{S}} A \rightsquigarrow p \qquad \Gamma; \Delta \vdash_{\text{ev}} p : A}{\Gamma; \Delta \vdash ?_A \Longrightarrow |p|}\ \text{R-Need}$$

$$\frac{\Gamma, x : A; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash \lambda(x : A).\, c \Longrightarrow \lambda(x : |A|).\, t}\ \text{R-Lambda} \qquad \frac{\Gamma, x : A; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash \mathsf{fix}\ x : A.\, c \Longrightarrow \mathsf{fix}\ x : |A|.\, t}\ \text{R-Fix}$$

$$\frac{\Gamma; (d : A) :: \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash \lambda\{d : A\}.\, c \Longrightarrow \lambda(d : |A|).\, t}\ \text{R-Implicit-Lambda} \qquad \frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash c_1\, c_2 \Longrightarrow t_1\, t_2}\ \text{R-App}$$

$$\frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash c_1\{c_2\} \Longrightarrow t_1\, t_2}\ \text{R-Implicit-App} \qquad \frac{\Gamma, \alpha; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash \Lambda\alpha.c \Longrightarrow \Lambda\alpha.t}\ \text{R-Type-Lambda}$$

$$\frac{\Gamma; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash c[A] \Longrightarrow t[|A|]}\ \text{R-Type-App} \qquad \frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma, x : A; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash \mathsf{let}\ x : A = c_1\ \mathsf{in}\ c_2 \Longrightarrow \mathsf{let}\ x = t_1\ \mathsf{in}\ t_2}\ \text{R-Let}$$

$$\frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; (d : A) :: \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash \mathsf{impl}\ d : A = c_1\ \mathsf{in}\ c_2 \Longrightarrow \mathsf{let}\ d = t_1\ \mathsf{in}\ t_2}\ \text{R-Impl}$$

$$\frac{\Gamma; \Delta \vdash c_j \Longrightarrow t_j \quad (j \in J)}{\Gamma; \Delta \vdash \{\ell_j = c_j\}_{j \in J} \Longrightarrow \{\ell_j = t_j\}_{j \in J}}\ \text{R-Record} \qquad \frac{\Gamma; \Delta \vdash c \Longrightarrow t}{\Gamma; \Delta \vdash c.\ell \Longrightarrow t.\ell}\ \text{R-Project}$$

$$\frac{}{\Gamma; \Delta \vdash n \Longrightarrow n}\ \text{R-Int} \qquad \frac{}{\Gamma; \Delta \vdash s \Longrightarrow s}\ \text{R-String} \qquad \frac{}{\Gamma; \Delta \vdash b \Longrightarrow b}\ \text{R-Bool}$$

$$\frac{\Gamma; \Delta \vdash c_1 \Longrightarrow t_1 \qquad \Gamma; \Delta \vdash c_2 \Longrightarrow t_2}{\Gamma; \Delta \vdash c_1 \odot c_2 \Longrightarrow t_1 \odot t_2}\ \text{R-BinOp}$$

7 Explicit target and independent typing

$$\begin{array}{rcl} T, U & ::= & \alpha \mid \mathsf{Int} \mid \mathsf{Bool} \mid \mathsf{String} \mid T \to U \mid \forall \alpha.T \mid \{\ell_j : T_j\}_{j \in J} \\ t & ::= & z \mid n \mid b \mid s \mid \lambda(z : T).\, t \mid t\, t \mid \Lambda\alpha.t \mid t[T] \\ & & \mid \mathsf{let}\ z = t\ \mathsf{in}\ t \mid \mathsf{fix}\ z : T.\, t \mid \{\ell_j = t_j\}_{j \in J} \mid t.\ell \mid t_1 \odot t_2 \\ H & ::= & \cdot \mid H, \alpha \mid H, z : T \end{array}$$

Target type formation is Section 1 with only the ordinary arrow. Every judgment assumes that H is well formed; all annotations and type arguments must be well formed in H. All record labels are distinct. The checker follows these syntax-directed rules to infer the unique type of a fully annotated term. Checking against an expected type first validates that type and then compares it with the inferred type by structural equality. The public boundary validates the whole input context, including unused bindings. In the indexed representation, its type-variable depth is nonnegative and every environment type is well scoped at that depth.

$$\frac{z : T \in H}{H \vdash z : T}\ \text{T-Var} \qquad \frac{H \vdash T\ \mathsf{type} \qquad H, z : T \vdash t : U}{H \vdash \lambda(z : T).\, t : T \to U}\ \text{T-Lambda}$$

$$\frac{H \vdash t_1 : T \to U \qquad H \vdash t_2 : T}{H \vdash t_1\, t_2 : U}\ \text{T-App} \qquad \frac{H, \alpha \vdash t : T \qquad \alpha \notin \mathrm{FV}(H)}{H \vdash \Lambda\alpha.t : \forall\alpha.T}\ \text{T-Type-Lambda}$$

$$\frac{H \vdash t : \forall\alpha.T \qquad H \vdash U\ \mathsf{type}}{H \vdash t[U] : T[U/\alpha]}\ \text{T-Type-App} \qquad \frac{H \vdash t_1 : T \qquad H, z : T \vdash t_2 : U}{H \vdash \mathsf{let}\ z = t_1\ \mathsf{in}\ t_2 : U}\ \text{T-Let}$$

$$\frac{H \vdash T\ \mathsf{type} \qquad H, z : T \vdash t : T}{H \vdash \mathsf{fix}\ z : T.\, t : T}\ \text{T-Fix} \qquad \frac{H \vdash t_j : T_j \quad (j \in J)}{H \vdash \{\ell_j = t_j\}_{j \in J} : \{\ell_j : T_j\}_{j \in J}}\ \text{T-Record}$$

$$\frac{H \vdash t : \{\ell_j : T_j\}_{j \in J} \qquad k \in J}{H \vdash t.\ell_k : T_k}\ \text{T-Project} \qquad \frac{n \in \mathbb{Z}}{H \vdash n : \mathsf{Int}}\ \text{T-Int} \qquad \frac{s\ \text{a string}}{H \vdash s : \mathsf{String}}\ \text{T-String}$$

$$\frac{b \in \{\mathsf{true}, \mathsf{false}\}}{H \vdash b : \mathsf{Bool}}\ \text{T-Bool} \qquad \frac{H \vdash t_1 : \arg(\odot) \qquad H \vdash t_2 : \arg(\odot)}{H \vdash t_1 \odot t_2 : \mathrm{res}(\odot)}\ \text{T-BinOp}$$

Phase invariant and index representation

Let $H = |\Gamma| \cup |\Delta|$ merge the two term environments using the same globally fresh names. The phase invariant is

$$\boxed{\Gamma; \Delta \vdash e \Rightarrow A \rightsquigarrow c \quad \wedge \quad \Gamma; \Delta \vdash c \Longrightarrow t \quad \Longrightarrow \quad H \vdash t : |A|.}$$

The reference implementation independently checks the final judgment; no mechanized metatheory is asserted here. The synthesizer is not a trusted source of typing derivations.

In the implementation, ordinary and instance references have independent de Bruijn indices. The resolver maintains one embedding for each into the current target environment. Every new term binder increments all previously embedded target indices. An ordinary binder, including a fixpoint binder, then extends only the Γ embedding; an implementation binder extends only the Δ embedding. A type binder weakens type indices in both environments and in embedded evidence. Type substitution is capture avoiding.

8 Call-by-value evaluation

$$\begin{array}{lcl} v & ::= & n \mid b \mid s \mid \{\ell_j = v_j\}_{j\in J} \mid \langle \lambda z.t, \rho\rangle \mid \langle \Lambda\alpha.t, \rho\rangle, \\ \rho & ::= & \cdot \mid \rho[z \mapsto v]. \end{array}$$

Evaluation ignores type annotations and type arguments. Type abstraction creates a delayed closure; type application forces it. Every record field is evaluated; their order is unspecified. Fields have no mutual scope and the calculus is pure, so well-typed record values, when evaluation terminates, do not depend on traversal order. Runtime records remain finite maps. Environments contain values; closures retain their defining environments. A fixpoint unfolds by substituting itself for its bound variable before evaluating the resulting term in the same environment. Substitution is capture avoiding under both term and type binders: free term references and type variables in the inserted term are weakened appropriately.

$$\frac{\rho(z) = v}{\rho \vdash z \Downarrow v}\ \textsc{V-Var} \qquad \frac{\rho \vdash t[(\mathsf{fix}\ z : T.\, t)/z] \Downarrow v}{\rho \vdash \mathsf{fix}\ z : T.\, t \Downarrow v}\ \textsc{V-Fix} \qquad \frac{}{\rho \vdash n \Downarrow n}\ \textsc{V-Int} \qquad \frac{}{\rho \vdash s \Downarrow s}\ \textsc{V-String}$$

$$\frac{}{\rho \vdash b \Downarrow b}\ \textsc{V-Bool} \qquad \frac{}{\rho \vdash \lambda(z : T).\, t \Downarrow \langle \lambda z.t, \rho\rangle}\ \textsc{V-Lambda} \qquad \frac{}{\rho \vdash \Lambda\alpha.t \Downarrow \langle \Lambda\alpha.t, \rho\rangle}\ \textsc{V-Type-Lambda}$$

$$\frac{\rho \vdash t_1 \Downarrow \langle \lambda z.t, \rho_0\rangle \qquad \rho \vdash t_2 \Downarrow v_2 \qquad \rho_0[z \mapsto v_2] \vdash t \Downarrow v}{\rho \vdash t_1\, t_2 \Downarrow v}\ \textsc{V-App}$$

$$\frac{\rho \vdash t_0 \Downarrow \langle \Lambda\alpha.t, \rho_0\rangle \qquad \rho_0 \vdash t \Downarrow v}{\rho \vdash t_0[T] \Downarrow v}\ \textsc{V-Type-App} \qquad \frac{\rho \vdash t_1 \Downarrow v_1 \qquad \rho[z \mapsto v_1] \vdash t_2 \Downarrow v}{\rho \vdash \mathsf{let}\ z = t_1\ \mathsf{in}\ t_2 \Downarrow v}\ \textsc{V-Let}$$

$$\frac{\rho \vdash t_j \Downarrow v_j \quad (j \in J)}{\rho \vdash \{\ell_j = t_j\}_{j\in J} \Downarrow \{\ell_j = v_j\}_{j\in J}}\ \textsc{V-Record} \qquad \frac{\rho \vdash t \Downarrow \{\ell_j = v_j\}_{j\in J} \qquad k \in J}{\rho \vdash t.\ell_k \Downarrow v_k}\ \textsc{V-Project}$$

$$\frac{\rho \vdash t_1 \Downarrow v_1 \qquad \rho \vdash t_2 \Downarrow v_2 \qquad \delta_\odot(v_1, v_2) = v}{\rho \vdash t_1 \odot t_2 \Downarrow v}\ \textsc{V-BinOp}$$

The evaluator implements these rules with an explicit continuation stack. Each frame stores the remaining premises and their immutable lexical environments: application evaluates its function and argument before the body, lets evaluate their initializer before the continuation, and records evaluate every field. Moving those continuations from the host stack to data preserves this order and each rule's result. Fixpoint unfolding is an evaluation transition. Its substitution traversal suspends child transformations and reconstructs each constructor once those children return, preserving the same substitution equations without consuming the host call stack. Inserted terms are shifted to preserve their free variables, then evaluated according to the ordinary rules.

Closed-program execution

Write $\Pi \rightsquigarrow_{\text{surface}} c : A$ for the sequential declaration and expression elaboration specified in Section 2, starting in empty environments.

$$\frac{\Pi \rightsquigarrow_{\text{surface}} c : A \qquad \cdot; [\,] \vdash c \Longrightarrow t \qquad \cdot \vdash t : |A| \qquad \cdot \vdash t \Downarrow v}{\Pi \Downarrow_{\mathcal{S}} v}\ \textsc{Execute}$$

There is no runtime evidence search. Resolving an implementation binding produces an ordinary target let, so its value can close over ordinary runtime variables. The compiled value remains first-class; registration affects only lexical evidence scope.